

Infosys Springboard Virtual Internship

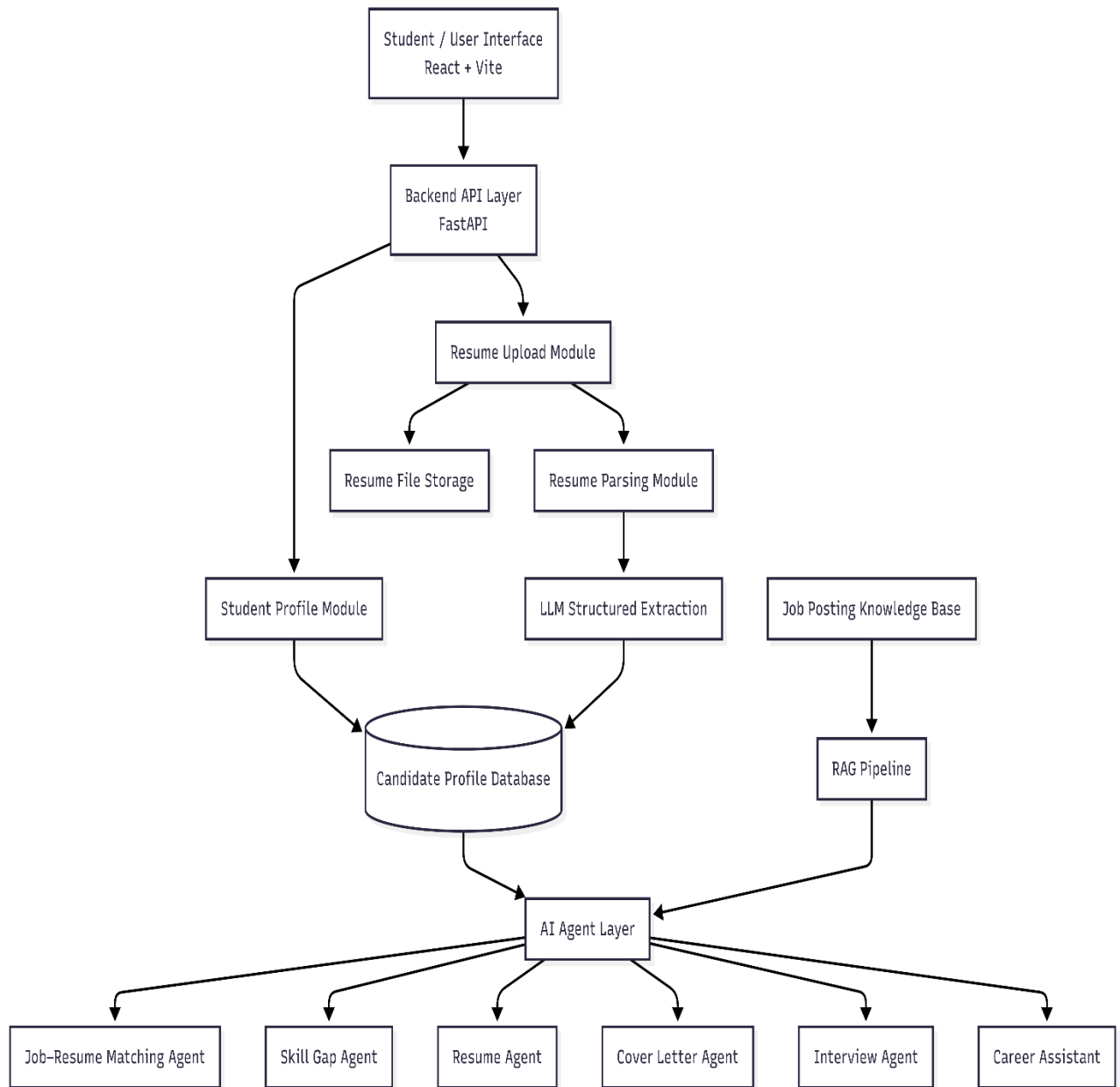
7.0

AI Career Companion Agent for Internship Matching and Interview Preparation

Milestone 1: Research and Technical Understanding

Project id: #M-3-1

Submitted by: Bibi Fatima Yadwad



1. Introduction

The **AI Career Companion Agent** is an AI-based system proposed to assist students throughout the internship application and preparation process. Applying for internships can be challenging because students need to understand job requirements, evaluate their eligibility, identify missing skills, customize their resumes, prepare cover letters, practice for interviews, and make informed career decisions.

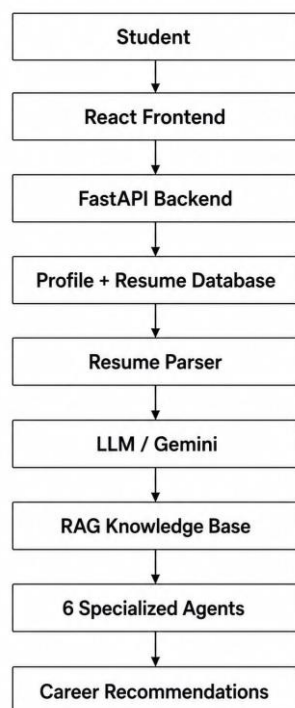
The proposed system aims to bring these activities together into a single intelligent platform. It collects a student's basic profile information and resume, extracts relevant information from the resume, converts the information into a

structured candidate profile, and uses this profile as the foundation for future AI-powered career assistance.

The system combines several technologies, including **React, FastAPI, SQLite, resume parsing libraries, Gemini/LLM-based information extraction, Retrieval-Augmented Generation (RAG), and specialized AI agents.**

The first milestone focuses on understanding the internship application workflow, researching RAG and multi-agent systems, designing the system architecture, defining the candidate database schema, selecting the technology stack, and establishing the resume-processing pipeline.

The overall proposed system follows this flow:



2. Internship Application Workflow

A typical internship application process consists of several stages. The AI Career Companion Agent is designed to provide assistance across these stages.

2.1 Student Profile Creation

The student first creates a profile containing basic information such as:

- Full name
- Email
- Phone number

- Location
- Target internship role
- LinkedIn profile

This information provides the basic identity and career preferences of the candidate.

2.2 Resume Upload

The student uploads a resume through the web interface. The system accepts common resume formats such as PDF, DOCX, and TXT.

The backend stores the resume metadata and maintains the association between the resume and the student's profile.

2.3 Resume Parsing

The uploaded resume is processed by a resume parser. The parser extracts readable text from the document.

For the proposed system:

- **PyPDF** can be used for PDF files.
- **python-docx** can be used for DOCX files.
- TXT files can be processed directly.

2.4 Structured Information Extraction

The extracted resume text is sent to an LLM such as Gemini. The LLM identifies important information and converts the unstructured resume into structured data.

The information includes:

- Summary
- Skills
- Education
- Experience
- Projects
- Certifications

2.5 Internship Matching

In later milestones, the structured candidate profile will be compared with internship job descriptions. The system will identify how closely the student's qualifications match the requirements of a particular internship.

2.6 Skill-Gap Analysis

The system can identify skills required by an internship that are missing or weak in the student's profile and suggest areas for improvement.

2.7 Resume and Cover-Letter Customization

The system can provide role-specific resume improvement suggestions and generate personalized cover-letter drafts.

2.8 Interview Preparation

The system can generate technical, behavioral, and role-specific interview questions and provide feedback on student answers.

2.9 Career Guidance

The final stage provides personalized career recommendations based on the student's profile, selected internship, skills, and preparation needs.

3. System Overview

The AI Career Companion Agent follows a layered architecture consisting of the **user interface, backend API layer, database and storage layer, resume-processing layer, LLM layer, RAG knowledge base, and AI agent layer**.

Each component has a specific responsibility while communicating with the other components through the backend.

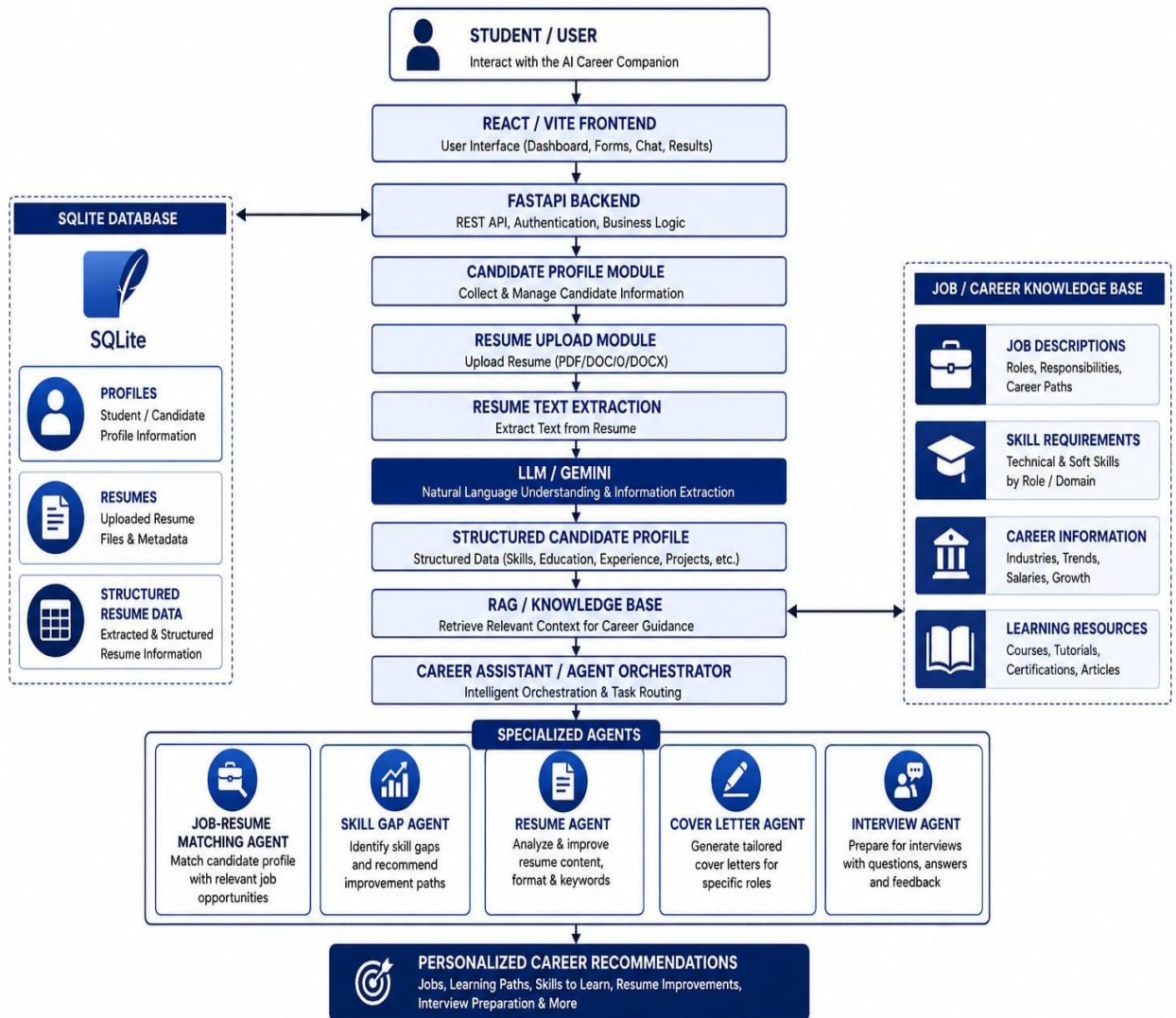
The major components are:

Component	Purpose
Student/User Interface	Allows students to create profiles, upload resumes, and view extracted information.
Backend/API Layer	Connects the frontend with the database, resume parser, LLM, and future AI agents.
Student Profile Module	Stores student information such as name, email, location, target role, and LinkedIn URL.

Resume Upload/Storage	Accepts resume files and stores their metadata and file location.
Resume Parsing Module	Extracts readable text from uploaded resumes.
LLM Structured Extraction	Converts resume text into structured skills, education, experience, projects, and certifications.
Candidate Profile Database	Stores profile information, resume metadata, and extracted resume information.
Job-Posting Knowledge Base	Stores job descriptions, internship requirements, and relevant company information.
RAG Pipeline	Retrieves relevant information from the knowledge base before generating AI responses.
AI Agent Layer	Contains six specialized agents for matching, skill-gap analysis, resume improvement, cover letters, interview preparation, and career guidance.

4. Proposed System Architecture

The high-level architecture describes how information moves from the student interface through the backend and AI-processing components.



4.1 Architecture Explanation

Student

The student is the primary user of the system. The student provides profile information and uploads a resume through the application.

React Frontend

The frontend is responsible for providing the user interface. **React with Vite** is selected because it provides a modular and fast development environment.

The frontend allows the student to:

- Enter profile information.
- Upload a resume.
- View extracted resume information.

- Interact with career-support features.
- View recommendations generated by the backend.

FastAPI Backend

FastAPI acts as the central API layer between the frontend, database, resume-processing modules, and AI components.

Its responsibilities include:

- Receiving API requests.
- Validating user input.
- Creating and updating student profiles.
- Handling resume uploads.
- Storing resume metadata.
- Calling the resume parser.
- Communicating with the LLM.
- Returning structured information to the frontend.
- Coordinating future AI-agent requests.

Profile and Resume Database

The database stores student profiles and information associated with uploaded resumes.

SQLite is selected for Milestone 1 because it is lightweight, simple to configure, and suitable for local development.

A production version can later migrate to **PostgreSQL**.

Resume Parser

The resume parser converts uploaded documents into machine-readable text.

The proposed libraries are:

- PyPDF for PDF files.
- python-docx for DOCX files.
- Direct text processing for TXT files.

LLM / Gemini

The extracted resume text is passed to an LLM such as Gemini. The model identifies relevant information and converts it into a structured representation.

For example, information such as:

Python, Java, SQL, TensorFlow

can be represented as:

```
{  
  "skills": [  
    "Python",  
    "Java",  
    "SQL",  
    "TensorFlow"  
  ]  
}
```

This structured representation allows the information to be reused by the future AI agents.

RAG Knowledge Base

The RAG knowledge base will contain information such as:

- Internship job descriptions.
- Required skills.
- Eligibility criteria.
- Company information.
- Interview preparation resources.
- Career resources.

The RAG system retrieves relevant information and provides it as context to the LLM or specialized agents.

AI Agent Layer

The AI agent layer contains six specialized agents. Each agent performs a specific career-support function.

The agents can use the student's structured profile and, when necessary, retrieve relevant information from the RAG knowledge base.

Career Recommendations

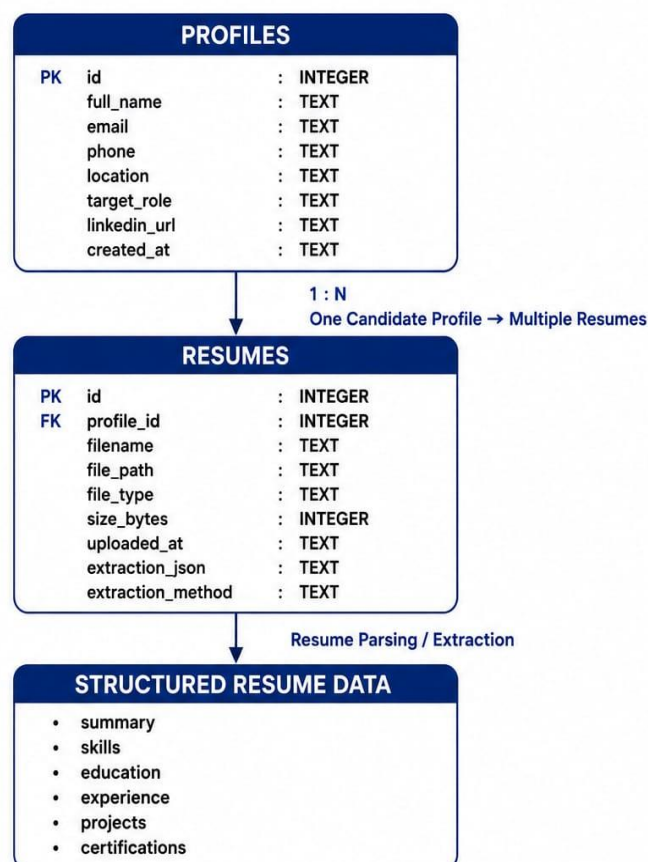
The final output is personalized career guidance based on the student's profile and the information retrieved from relevant sources.

5. Candidate Profile and Database Design

The database design establishes how student profiles and resumes are stored and related.

The main relationship is between the **Profiles** and **Resumes** entities.

Candidate Profile & Resume Database Schema



5.1 Profiles Table

The **Profiles** table stores the basic information of each student.

Field	Description
id	Primary key and unique identifier for the student profile
full_name	Student's full name
email	Student's email address
phone	Student's contact number
location	Student's city or preferred location
target_role	Internship or career role targeted by the student
linkedin_url	Student's LinkedIn profile URL
created_at	Date and time when the profile was created

5.2 Resumes Table

The **Resumes** table stores metadata and extracted information related to uploaded resumes.

Field	Description
id	Primary key identifying the resume
profile_id	Foreign key connecting the resume to a student profile
filename	Name of the uploaded resume
file_path	Location where the uploaded file is stored
file_type	Resume format such as PDF, DOCX, or TXT
size_bytes	Size of the uploaded file
uploaded_at	Date and time of upload
extraction_json	Structured information extracted from the resume
extraction_method	Method or LLM used to perform the extraction

The relationship between **Profiles** and **Resumes** is **one-to-many (1:N)**.

This means that one student profile can have multiple uploaded resumes, while each resume belongs to one student profile.

5.3 Structured Resume

After processing, the resume information is represented using structured fields.

Summary

Contains a concise representation of the student's professional or academic profile.

Skills

Contains technical and soft skills identified from the resume.

Education

Contains details such as:

- Degree
- Institution
- Graduation year
- Academic information

Experience

Contains internship, employment, volunteer, or other relevant professional experience.

Projects

Contains academic, personal, or professional projects and their relevant technologies.

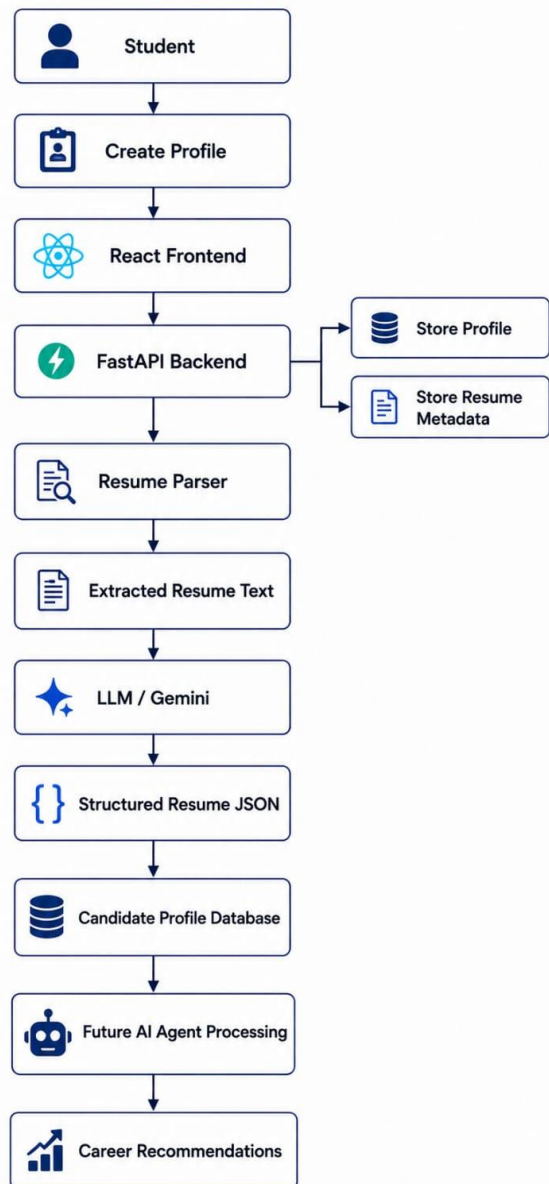
Certifications

Contains relevant certifications, courses, and credentials.

This structured representation is important because future AI agents can directly use the extracted information instead of repeatedly processing the original resume.

6. Data Flow

The system's initial data flow begins with profile creation and ends with a structured candidate profile.



Step-by-Step Explanation

Step 1 – Profile Creation

The student enters basic information through the React interface.

Step 2 – Backend Processing

The frontend sends the information to the FastAPI backend. The backend validates the data.

Step 3 – Database Storage

The profile information is stored in SQLite.

Step 4 – Resume Upload

The student uploads a PDF, DOCX, or TXT resume.

Step 5 – Resume Parsing

The backend passes the uploaded resume to the appropriate parser, which extracts its readable text.

Step 6 – LLM Processing

The extracted text is provided to Gemini/LLM for structured information extraction.

Step 7 – Structured Profile Creation

The extracted skills, education, experience, projects, certifications, and summary are organized into structured JSON.

Step 8 – Storage

The structured information is associated with the student's resume and stored in the database.

Step 9 – Future AI Processing

In later milestones, the candidate profile will be combined with relevant internship information retrieved through the RAG pipeline.

Step 10 – Recommendations

The appropriate AI agent will analyze the information and provide personalized career recommendations.

7. Retrieval-Augmented Generation (RAG)

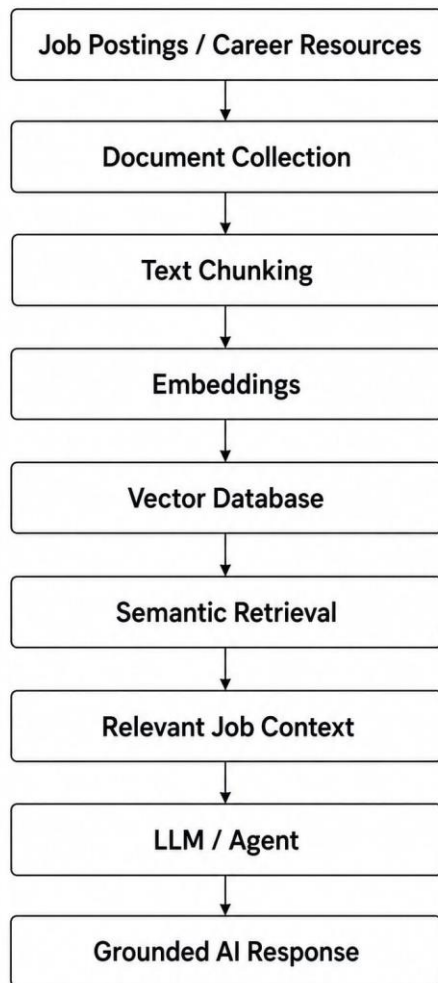
7.1 Understanding RAG

Retrieval-Augmented Generation, or **RAG**, is an AI technique that combines an LLM with an external knowledge source.

A language model may not contain the latest information about internship openings, company requirements, or changing job descriptions. RAG addresses this limitation by retrieving relevant information from an external knowledge base before generating an answer.

For the AI Career Companion Agent, the RAG knowledge base can contain internship descriptions, job requirements, company information, and career resources.

7.2 RAG Workflow



The major steps are:

1. Collect job postings and career documents.
2. Divide documents into smaller chunks.
3. Convert chunks into embeddings.
4. Store embeddings in a vector database.
5. Retrieve relevant information based on the student's query.
6. Provide the retrieved information to the LLM.
7. Generate a grounded and relevant response.

Potential technologies for the future RAG layer include **Chroma** or **pgvector**.

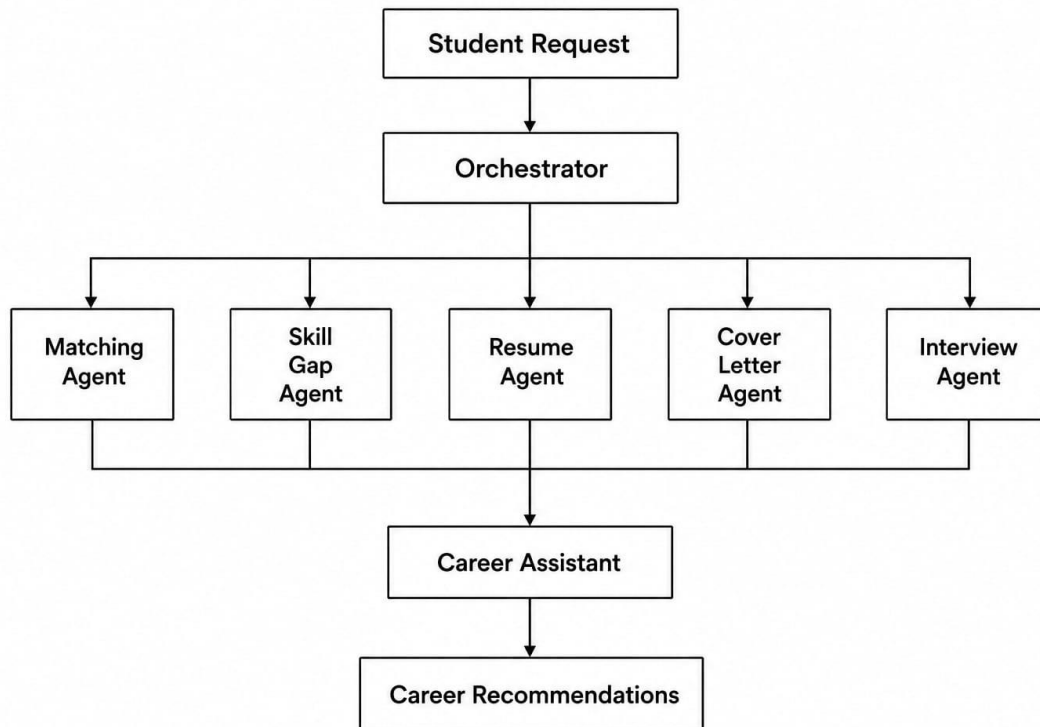
8. Multi-Agent System Design

A multi-agent system divides a complex task among multiple specialized AI agents.

For the AI Career Companion Agent, six agents are proposed.

Agent	Responsibility
Job–Resume Matching Agent	Compares the student's profile with a job description and provides a match score and explanation.
Skill Gap Agent	Identifies missing or weak skills and recommends areas for improvement.
Resume Agent	Suggests improvements to resume content for a selected job role.
Cover Letter Agent	Creates a personalized cover-letter draft based on the student's profile and job description.
Interview Agent	Generates interview questions and provides practice feedback.
Career Assistant	Answers general career and internship-related questions and guides the student through the platform.

8.1 Agent Workflow



The backend or orchestrator determines which agent should handle the student's request.

For example:

- A job suitability question can be handled by the **Job–Resume Matching Agent**.
- A missing-skills question can be handled by the **Skill Gap Agent**.
- A resume improvement request can be handled by the **Resume Agent**.
- An interview request can be handled by the **Interview Agent**.

This separation makes the system modular and easier to maintain.

9. AI Agent Responsibilities

9.1 Job–Resume Matching Agent

The Job–Resume Matching Agent compares the structured candidate profile with the requirements of a selected internship.

It can consider:

- Required technical skills.

- Preferred skills.
- Education.
- Experience.
- Projects.
- Certifications.

The agent can generate a match score along with an explanation of the matching and missing requirements.

9.2 Skill Gap Agent

The Skill Gap Agent identifies the difference between the student's current skills and the skills required for a target internship.

For example, if an internship requires Python, SQL, Git, FastAPI, and REST APIs, while the student has Python, SQL, and Git, the agent can identify FastAPI and REST APIs as skill gaps.

It can then suggest which skills should be prioritized.

9.3 Resume Agent

The Resume Agent analyzes the student's resume in relation to a selected internship.

It can suggest:

- Relevant skills to highlight.
- Projects that should receive greater emphasis.
- Improvements to project descriptions.
- Better alignment with job requirements.
- Removal of unnecessary information.

The system should make recommendations based on the student's actual qualifications and should not invent experience or skills.

9.4 Cover Letter Agent

The Cover Letter Agent generates a personalized cover-letter draft using:

- Student profile.
- Structured resume.

- Selected internship.
- Job description.
- Relevant projects and skills.

9.5 Interview Agent

The Interview Agent supports interview preparation by generating:

- Technical questions.
- Behavioral questions.
- Project-based questions.
- Role-specific questions.
- Follow-up questions.

It can also provide feedback on student responses.

9.6 Career Assistant

The Career Assistant handles general career-related questions.

Examples include:

- Which skills should I learn next?
- How should I prepare for this internship?
- Which projects should I highlight?
- How can I improve my interview preparation?

The Career Assistant can also guide the student toward the specialized agent that is most appropriate for the request.

10. Technology Stack

Layer	Technology	Purpose
Frontend	React + Vite	Building the student-facing web interface
Backend	FastAPI + Python	API development, validation, file handling, and system coordination
Database	SQLite	Local storage for Milestone 1

Future Database	PostgreSQL	Scalable production database
Resume Parsing	PyPDF + python-docx	Extracting text from PDF and DOCX resumes
LLM	Gemini	Structured resume extraction and future AI interactions
File Storage	Local Storage	Storing uploaded resumes during development
Future RAG Store	Chroma or pgvector	Semantic search over job and career information
Version Control	Git + GitHub	Source-code management and collaboration

11. Technology Selection Rationale

React + Vite

React provides a component-based approach for developing the frontend, while Vite provides a fast development environment.

FastAPI

FastAPI is suitable for developing the backend because it supports API creation, request validation, file uploads, and integration with Python-based AI libraries.

SQLite

SQLite is lightweight and requires minimal configuration, making it suitable for the first milestone.

PyPDF and python-docx

These libraries provide a straightforward method for extracting text from common resume formats.

Gemini / LLM

An LLM can understand unstructured resume text and transform it into structured information. It can also support future career-related AI tasks.

Chroma / pgvector

A vector database will be useful for the future RAG implementation because it can store embeddings and support semantic retrieval.

Git and GitHub

Git and GitHub provide version control, collaboration, and project history management.

12. Milestone 1 Scope

The primary objective of Milestone 1 is to establish a reliable **candidate-profile and resume-processing foundation**.

The milestone focuses on:

1. Student profile creation.
2. Profile data validation.
3. Profile storage.
4. Resume upload.
5. Resume metadata storage.
6. Local resume file storage.
7. PDF parsing.
8. DOCX parsing.
9. TXT processing.
- 10.Resume text extraction.
- 11.Gemini/LLM-based structured extraction.
- 12.Extraction of skills.
- 13.Extraction of education.
- 14.Extraction of experience.
- 15.Extraction of projects.
- 16.Extraction of certifications.
- 17.Structured resume JSON storage.
- 18.Database schema design.

19. Overall AI system architecture design.

20. Research into RAG and multi-agent systems.

The full RAG pipeline and six specialized AI agents are part of the overall system architecture but will be implemented progressively in later milestones.

13. Future Implementation

After the completion of the initial candidate-profile foundation, the following features can be added:

RAG Knowledge Base

A searchable knowledge base containing internship descriptions, requirements, company information, and career resources.

Internship Matching

Automatic comparison of student profiles with available internship opportunities.

Skill-Gap Analysis

Identification of missing skills and personalized learning recommendations.

Resume Customization

Role-specific resume improvement suggestions.

Cover-Letter Generation

Personalized cover-letter generation based on the selected internship.

Interview Preparation

AI-powered mock interviews and personalized interview feedback.

Application Tracking

Tracking application status such as:

- Saved
- Applied
- Assessment
- Interview
- Selected

- Rejected

Scalable Deployment

The initial SQLite database can be migrated to PostgreSQL, and local file storage can be replaced with cloud storage when the application moves toward production.

14. Expected Benefits

The proposed system is expected to provide the following benefits:

Personalized Career Support

The system uses the student's own profile and resume to provide more relevant guidance.

Reduced Manual Work

Students do not need to manually repeat resume analysis and skill comparison for every internship.

Better Understanding of Job Requirements

The matching and skill-gap components can explain how the student's profile relates to a particular opportunity.

Modular AI Architecture

Specialized agents allow individual career-support functions to be developed and improved independently.

Grounded Responses

The future RAG pipeline can provide relevant job information to the AI system rather than relying only on general model knowledge.

Extensibility

The architecture can be expanded with additional agents, job sources, databases, and career-support features.

15. Conclusion

The research and technical analysis establish a clear foundation for the development of the **AI Career Companion Agent for Internship Matching and Interview Preparation**.

The proposed architecture begins with the student-facing React interface and uses FastAPI as the central backend layer. Student profiles and resume metadata are stored in SQLite during Milestone 1. Uploaded resumes are processed using appropriate parsing libraries, and the extracted text is passed to Gemini/LLM for conversion into structured information such as skills, education, experience, projects, and certifications.

The structured candidate profile is the core foundation of the system because it can be reused by multiple future AI components. The proposed RAG pipeline will provide access to relevant internship and career information, while the multi-agent architecture divides career-support responsibilities among six specialized agents: **Job–Resume Matching, Skill Gap, Resume, Cover Letter, Interview, and Career Assistant.**

Milestone 1 therefore concentrates on building and validating the candidate-profile and resume-processing pipeline before introducing the complete RAG and multi-agent functionality. This staged approach reduces implementation complexity while providing a clear path toward the complete system.

Overall, the proposed architecture provides a modular, extensible, and practical foundation for an intelligent career-support platform that can eventually assist students throughout the internship journey—from **profile creation and resume analysis to internship matching, skill development, resume customization, cover-letter preparation, interview practice, and personalized career recommendations.**